AI 500 PA2 - Tutorial Handout

Unsupervised Learning — Implementation Guide

Contents

1 Task 1.1: K-means++ Initialization, K-means, and K-medoids **1**
- 1.1 Part A: Useful NumPy Functions for K-means++ 1
- 1.2 Part B: Useful NumPy Functions for K-means 3
- 1.3 Part C: Useful NumPy Functions for K-medoids 5

2 Task 1.2: Within-Cluster Sum of Squares (WCSS) **6**
- 2.1 Useful NumPy Functions for WCSS 6

3 Task 2.2: DBSCAN Implementation **8**
- 3.1 Useful NumPy Functions for DBSCAN 8

4 Task 3.1: Gaussian Mixture Model with EM Algorithm **9**
- 4.1 Part A: Useful NumPy Functions for Gaussian PDF 9
- 4.2 Part B: Useful NumPy Functions for GMM EM 11

Overview

This notebook provides:

1. **Useful NumPy functions** with examples for each task.
2. **Algorithmic intuition** for implementing the 5 coding tasks.

Use this as a reference while working on your assignment. The examples use simple dummy data to illustrate how each function works.

---

1 Task 1.1: K-means++ Initialization, K-means, and K-medoids

1.1 Part A: Useful NumPy Functions for K-means++

```
import numpy as np

# ============================================
# Function 1: np.random.randint()
# Purpose: Pick a random starting point
```

```python
# ============================================
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
n = X.shape[0]
random_idx = np.random.randint(n)
print(f"Random index: {random_idx}")
print(f"Random point: {X[random_idx]}")

# ============================================
# Function 2: np.linalg.norm() with broadcasting
# Purpose: Compute distances from one point to all points
# ============================================
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
point = np.array([2, 3])

# Compute distances from 'point' to all points in X
distances = np.linalg.norm(X - point, axis=1)
print(f"Distances: {distances}")

# ============================================
# Function 3: Squaring arrays (element-wise)
# Purpose: Square distances for probability calculation
# ============================================
distances = np.array([1.4, 2.8, 4.2, 6.0])
squared_distances = distances ** 2
print(f"Squared distances: {squared_distances}")

# ============================================
# Function 4: np.minimum() - element-wise minimum
# Purpose: Keep track of minimum distance to nearest centroid
# ============================================
current_min_dist = np.array([10.0, 8.0, 12.0, 5.0])
new_distances = np.array([7.0, 9.0, 6.0, 3.0])

updated_min_dist = np.minimum(current_min_dist, new_distances)
print(f"Updated minimum distances: {updated_min_dist}")

# ============================================
# Function 5: np.random.choice() with probabilities
# Purpose: Sample next centroid with weighted probability
# ============================================
n = 5
distances = np.array([1.0, 4.0, 9.0, 16.0, 25.0])
probabilities = distances / distances.sum()  # Normalize to sum to 1

print(f"Probabilities: {probabilities}")

# Sample one index according to these probabilities
chosen_idx = np.random.choice(n, p=probabilities)
print(f"Chosen index: {chosen_idx}")
```

Listing 1: K-means++ helper examples

K-means++ Algorithmic Intuition

1. **Initialize storage**: Create an array to hold k centroids.
2. **First centroid**: Pick one point randomly from X.
3. **For each remaining centroid** (repeat $k-1$ times):
   - Compute squared distance from each point to its nearest existing centroid.
   - Track minimum distances (use `np.minimum` to update).
   - Convert distances to probabilities (divide by sum).
   - Sample next centroid with probability proportional to squared distance.
4. **Return**: Array of k initial centroids.

Key insight: Points far from existing centroids have higher probability of being chosen next; this spreads centroids across the data.

1.2 Part B: Useful NumPy Functions for K-means

```python
# ============================================
# Function 1: Broadcasting for distance matrix
# Purpose: Compute distances from all points to all centroids
# ============================================
X = np.array([[1, 2], [2, 3], [8, 9], [9, 10]])  # 4 points
centroids = np.array([[1.5, 2.5], [8.5, 9.5]])   # 2 centroids

# Add new axis to X: shape (4, 2) -> (4, 1, 2)
X_expanded = X[:, np.newaxis]
print(f"X_expanded shape: {X_expanded.shape}")

# Broadcasting: (4, 1, 2) - (2, 2) -> (4, 2, 2)
diff = X_expanded - centroids
print(f"Difference array shape: {diff.shape}")

# Compute distances: norm along axis=2 gives (4, 2)
distances = np.linalg.norm(diff, axis=2)
print(f"Distance matrix shape: {distances.shape}")
print(f"Distance matrix:\n{distances}")

# ============================================
# Function 2: np.argmin() for cluster assignment
# Purpose: Find nearest centroid for each point
# ============================================
distances = np.array([[2.1, 10.5],
                      [3.0, 9.8],
                      [11.2, 1.4],
                      [12.0, 2.1]])

labels = np.argmin(distances, axis=1)
print(f"Cluster assignments: {labels}")

```

```python
# ============================================
# Function 3: Boolean indexing for cluster filtering
# Purpose: Get all points belonging to a specific cluster
# ============================================
X = np.array([[1, 2], [2, 3], [8, 9], [9, 10]])
labels = np.array([0, 0, 1, 1])

cluster_0_points = X[labels == 0]
print(f"Points in cluster 0:\n{cluster_0_points}")

# ============================================
# Function 4: np.mean() for centroid update
# Purpose: Compute new centroid as mean of cluster points
# ============================================
cluster_points = np.array([[1, 2], [2, 3], [1.5, 2.5]])
new_centroid = cluster_points.mean(axis=0)
print(f"New centroid: {new_centroid}")

# ============================================
# Function 5: np.allclose() for convergence check
# Purpose: Check if centroids have stopped changing
# ============================================
old_centroids = np.array([[1.5, 2.5], [8.5, 9.5]])
new_centroids = np.array([[1.500001, 2.499999], [8.500002, 9.499998]])

converged = np.allclose(old_centroids, new_centroids)
print(f"Converged: {converged}")

# ============================================
# Function 6: np.any() to check for empty clusters
# Purpose: Handle edge case where a cluster has no points
# ============================================
labels = np.array([0, 0, 1, 1, 2, 2])
k = 4  # but we only have clusters 0,1,2

for i in range(k):
    if np.any(labels == i):
        print(f"Cluster {i} exists")
    else:
        print(f"Cluster {i} is empty - keep old centroid")
```

Listing 2: K-means helper examples

K-means Algorithmic Intuition

1. **Initialize centroids**:
   - If `init='kmeanspp'`: use `kmeanspp_init`
   - Otherwise: randomly sample k points from X
2. **Iterate until convergence** (up to `max_iter`):
   - **Assignment step**: compute distance matrix (broadcasting), assign each point to nearest centroid (`np.argmin`).

- **Update step**: for each cluster, compute new centroid as mean of members (boolean indexing). If cluster empty, keep old centroid.
- **Check convergence**: if centroids haven't changed (`np.allclose`) break early.

3. **Return**: final labels and centroids.

Key insight: K-means minimizes within-cluster variance by alternating assignment and update.

1.3 Part C: Useful NumPy Functions for K-medoids

```python
# ============================================
# Function 1: np.where() for cluster indices
# Purpose: Get indices of all points in a cluster
# ============================================
labels = np.array([0, 1, 0, 2, 1, 0, 2])
cluster_0_indices = np.where(labels == 0)[0]
print(f"Indices of points in cluster 0: {cluster_0_indices}")

# ============================================
# Function 2: Pairwise distance matrix
# Purpose: Compute distances between all pairs in a cluster
# ============================================
cluster_points = np.array([[1, 2], [2, 3], [1.5, 2.2]])

expanded = cluster_points[:, np.newaxis]
pairwise_diff = expanded - cluster_points
pairwise_distances = np.linalg.norm(pairwise_diff, axis=2)

print(f"Pairwise distance matrix:\n{pairwise_distances}")

# ============================================
# Function 3: np.sum() along axis for total distances
# Purpose: Find point with minimum sum of distances
# ============================================
pairwise_distances = np.array([[0.0, 1.4, 0.5],
                               [1.4, 0.0, 0.9],
                               [0.5, 0.9, 0.0]])

sum_distances = np.sum(pairwise_distances, axis=1)
print(f"Sum of distances for each point: {sum_distances}")

best_medoid_idx = np.argmin(sum_distances)
print(f"Best medoid (minimum sum): index {best_medoid_idx}")

# ============================================
# Function 4: Set comparison for convergence
# Purpose: Check if medoid indices have changed
# ============================================
old_medoid_indices = [0, 5, 10]
new_medoid_indices = [0, 5, 10]

converged = set(old_medoid_indices) == set(new_medoid_indices)
```

```
print(f"Medoids unchanged: {converged}")

new_medoid_indices = [0, 6, 10]
converged = set(old_medoid_indices) == set(new_medoid_indices)
print(f"Medoids unchanged: {converged}")  # False
```

Listing 3: K-medoids helper examples

K-medoids Algorithmic Intuition

1. **Initialize medoids**: randomly select k point indices from dataset (store indices).
2. **Iterate until convergence**:
   - **Assignment**: compute distances from all points to medoids and assign each point to nearest medoid.
   - **Update**: for each non-empty cluster, compute pairwise distances among points, sum distances per point, choose point with minimum sum as new medoid.
   - **Check convergence**: if medoid indices unchanged (set comparison) break.
3. **Return**: final labels and medoid points.

Key difference from K-means: medoids are actual data points (robust to outliers).

---

2 Task 1.2: Within-Cluster Sum of Squares (WCSS)

2.1 Useful NumPy Functions for WCSS

```
# ============================================
# Function 1: np.unique() to get cluster count
# Purpose: Find how many unique clusters exist
# ============================================
labels = np.array([0, 1, 0, 2, 1, 0, 2, 2])
unique_labels = np.unique(labels)
n_clusters = len(unique_labels)
print(f"Number of clusters: {n_clusters}")
print(f"Cluster IDs: {unique_labels}")

# ============================================
# Function 2: Boolean indexing for cluster filtering
# Purpose: Get points belonging to a specific cluster
# ============================================
X = np.array([[1, 2], [2, 3], [8, 9], [9, 10], [8.5, 9.5]])
labels = np.array([0, 0, 1, 1, 1])
centroids = np.array([[1.5, 2.5], [8.5, 9.5]])

cluster_1_points = X[labels == 1]
print(f"Points in cluster 1:\n{cluster_1_points}")

# ============================================
# Function 3: np.square() for squared differences
```

```python
# Purpose: Compute squared Euclidean distance
# ============================================
points = np.array([[8, 9], [9, 10], [8.5, 9.5]])
center = np.array([8.5, 9.5])

differences = points - center
print(f"Differences:\n{differences}")

squared_diff = np.square(differences)
print(f"Squared differences:\n{squared_diff}")

squared_distances = np.sum(squared_diff, axis=1)
print(f"Squared distances: {squared_distances}")

# ============================================
# Function 4: Alternative - direct squared distance
# Purpose: Compute squared distances more concisely
# ============================================
squared_distances_v1 = np.linalg.norm(points - center, axis=1) ** 2
squared_distances_v2 = np.sum((points - center) ** 2, axis=1)
print(f"Method 1: {squared_distances_v1}")
print(f"Method 2: {squared_distances_v2}")

# ============================================
# Function 5: np.sum() for accumulating WCSS
# Purpose: Sum up squared distances within a cluster
# ============================================
squared_distances = np.array([0.5, 1.0, 0.25, 2.0])
cluster_wcss = np.sum(squared_distances)
print(f"WCSS for this cluster: {cluster_wcss}")
```

Listing 4: WCSS helper examples

WCSS Algorithmic Intuition

1. **Initialize**: determine number of clusters (unique labels), set `wcss`=0.0.
2. **For each cluster** i:
   - Get cluster points (boolean indexing). If empty, skip.
   - Get center (centroid or medoid).
   - Compute squared distances from each point to center and sum.
   - Accumulate into total WCSS.
3. **Return**: total WCSS value.

Key insight: WCSS measures within-cluster variance. It decreases as k increases.

---

3 Task 2.2: DBSCAN Implementation

3.1 Useful NumPy Functions for DBSCAN

```python
# ============================================
# Function 1: np.full() for initialization
# Purpose: Initialize all labels to -1 (noise)
# ============================================
n = 10
labels = np.full(n, -1)
print(f"Initial labels: {labels}")

# ============================================
# Function 2: Distance calculation for region query
# Purpose: Find all neighbors within eps distance
# ============================================
X = np.array([[0, 0], [1, 0], [0.5, 0.5], [10, 10], [10.5, 10.5]])
query_point_idx = 0
query_point = X[query_point_idx]

# Compute distances from query point to all points
distances = np.linalg.norm(X - query_point, axis=1)
print(f"Distances from point {query_point_idx}: {distances}")

eps = 1.5
neighbors = np.where(distances <= eps)[0]
print(f"Neighbors within eps={eps}: {neighbors}")

# ============================================
# Function 3: np.where() for finding neighbors
# Purpose: Get indices of points satisfying a condition
# ============================================
distances = np.array([0.0, 1.0, 0.7, 14.1, 14.5])
eps = 1.5
neighbor_indices = np.where(distances <= eps)[0]
print(f"Neighbor indices: {neighbor_indices}")

# ============================================
# Function 4: np.concatenate() for expanding neighbors
# Purpose: Add new neighbors to the expansion queue
# ============================================
current_neighbors = np.array([1, 2, 3])
new_neighbors = np.array([5, 6])
expanded_neighbors = np.concatenate((current_neighbors, new_neighbors))
print(f"Expanded neighbor list: {expanded_neighbors}")

# ============================================
# Function 5: Working with nonlocal variables
# Purpose: Modify cluster_id from nested function
# ============================================
def dbscan_example():
    cluster_id = 0  # Outer scope variable

    def expand_cluster():
```

```
        nonlocal cluster_id  # Allows modification
        print(f"Current cluster_id: {cluster_id}")
        cluster_id += 1
        print(f"Updated cluster_id: {cluster_id}")

    expand_cluster()
    print(f"Final cluster_id: {cluster_id}")

dbscan_example()
```

Listing 5: DBSCAN helper examples

DBSCAN Algorithmic Intuition

Goal: find clusters by density (core points, border points, noise).

Core point has at least `min_samples` neighbors within `eps`.

Border point is within `eps` of a core point, but not itself core.

Noise is neither core nor border (label = -1).

Algorithm structure for `dbscan(X, eps, min_samples)`:

1. Initialize labels to -1 and `cluster_id = 0`.
2. Define `region_query(point_idx)`: return indices with distance $\leq$ `eps`.
3. Define `expand_cluster(point_idx, neighbors)`:
   - Mark point as belonging to current cluster.
   - Use a `while` loop to allow neighbor list to grow while iterating.
   - Add neighbors of core neighbors to expansion queue (via concatenation).
4. Main loop: for each point, if unprocessed, query neighbors; if enough neighbors, expand cluster and increment `cluster_id`.
5. Return labels.

Key insight: DBSCAN discovers clusters by density; no need to specify k.

---

4 Task 3.1: Gaussian Mixture Model with EM Algorithm

4.1 Part A: Useful NumPy Functions for Gaussian PDF

```
# ============================================
# Function 1: np.linalg.inv() - Matrix inverse
# Purpose: Compute inverse of covariance matrix
# ============================================
cov = np.array([[2.0, 0.5],
                [0.5, 1.0]])
inv_cov = np.linalg.inv(cov)
print(f"Inverse covariance:\n{inv_cov}")
```

```python

# ================================================
# Function 2: np.linalg.det() - Matrix determinant
# Purpose: Compute determinant of covariance matrix
# ================================================
det = np.linalg.det(cov)
print(f"Determinant: {det}")

# ================================================
# Function 3: Matrix multiplication with element-wise product
# Purpose: Efficiently compute (x-  )^T   ^{-1} (x-  ) for all points
# ================================================
X = np.array([[1, 2], [3, 4], [5, 6]])
mean = np.array([3, 4])
inv_cov = np.array([[1.0, 0.2],
                    [0.2, 0.8]])

diff = X - mean
temp = diff @ inv_cov
squared_term = temp * diff
mahalanobis_squared = np.sum(squared_term, axis=1)
print(f"Mahalanobis squared distances: {mahalanobis_squared}")

# ================================================
# Function 4: np.exp() for exponential
# Purpose: Compute e^(-0.5 * value)
# ================================================
exponent_values = np.array([0.5, 1.0, 2.0, 5.0])
exp_result = np.exp(-0.5 * exponent_values)
print(f"Exponential results: {exp_result}")

# ================================================
# Function 5: np.eye() for regularization
# Purpose: Add small value to diagonal to prevent singular matrix
# ================================================
d = 3
regularization = np.eye(d) * 1e-6
print(f"Regularization matrix:\n{regularization}")
```

Listing 6: Gaussian PDF helper examples

Gaussian PDF Algorithmic Intuition

Goal: compute multivariate Gaussian density.

$$p(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}\,|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big)$$

Algorithm structure for `gaussian_pdf(X, mean, cov)`:

1. Get d (number of features).
2. Compute `diff = X - mean`.

3. Compute `inv_cov = np.linalg.inv(cov)` (add regularization if needed).
4. Compute exponent term using matrix multiply then element-wise product and sum.
5. Compute determinant $\det(\Sigma)$.
6. Combine normalization constant and exponential to get densities.
7. Return densities.

Why regularize? Add a small value (e.g. 1e−6) to the diagonal to avoid singular covariance.

4.2 Part B: Useful NumPy Functions for GMM EM

```
# ================================================
# Function 1: np.ones() / np.random.choice() for initialization
# ================================================
n, d, k = 100, 2, 3
weights = np.ones(k) / k
X = np.random.randn(n, d)
means = X[np.random.choice(n, k, replace=False)]

# ================================================
# Function 2: np.cov() for covariance
# ================================================
X = np.array([[1, 2], [2, 3], [1.5, 2.5], [3, 4]])
cov_matrix = np.cov(X.T)

# ================================================
# Function 3: Broadcasting for E-step
# ================================================
n, k = 5, 3
lik = np.random.rand(n, k)
weights = np.array([0.3, 0.5, 0.2])
weighted_lik = lik * weights

# ================================================
# Function 4: Normalization for responsibilities
# ================================================
weighted_lik = np.array([[0.3, 0.5, 0.2],
                         [0.1, 0.8, 0.1],
                         [0.6, 0.3, 0.1]])
sum_weighted = weighted_lik.sum(axis=1)[:, np.newaxis]
responsibilities = weighted_lik / sum_weighted

# ================================================
# Function 5: Weighted sum for M-step
# ================================================
X = np.array([[1, 2], [2, 3], [3, 4], [4, 5]])
resp = np.array([[0.9, 0.1],
                 [0.8, 0.2],
                 [0.2, 0.8],
                 [0.1, 0.9]])

```

```
resp_0 = resp[:, 0]
nk_0 = resp_0.sum()
mean_0 = (resp_0 @ X) / nk_0

# ============================================
# Function 6: Outer product for covariance update
# ============================================
X = np.array([[1, 2], [2, 3], [3, 4]])
mean = np.array([2, 3])
resp = np.array([0.9, 0.8, 0.2])
diff = X - mean
weighted_diff = resp[:, np.newaxis] * diff
cov = weighted_diff.T @ diff / resp.sum()

# ============================================
# Function 7: np.log() and np.sum() for convergence
# ============================================
weighted_lik_sum = np.array([0.8, 0.9, 0.5, 1.2])
log_lik = np.sum(np.log(weighted_lik_sum))
```

Listing 7: GMM / EM helper examples

GMM EM Algorithmic Intuition

Goal: fit a mixture of Gaussians using EM.

1. **Initialize** weights (uniform), means (random sample), covariances (data covariance + reg).
2. **EM loop** up to `max_iter`:
   - **E-step**: compute likelihoods per component, weight them by current weights, normalize to get responsibilities.
   - **M-step**: compute effective counts $n_k = \sum_i r_{ik}$; update weights $= n_k/n$; update means as weighted averages; update covariances using weighted outer products and add regularization.
   - **Convergence check**: compute log-likelihood $\log L = \sum_i \log(\sum_k w_k p_k(x_i))$. Stop if change $<$ tol.
3. **Return**: weights, means, covariances.

Key insights:

- E-step: soft assignments (probabilities).
- M-step: parameter updates using soft counts.
- Regularize covariances to avoid singular matrices.

Summary: Algorithmic Patterns

Pattern 1: Iterative Assignment–Update

Used in: K-means, K-medoids.

1. Assign points to clusters.
2. Update cluster centers (mean or medoid).
3. Repeat until convergence.

Pattern 2: Density-Based Expansion

Used in: DBSCAN.

1. Find core points (dense regions).
2. Expand clusters by density reachability.
3. Mark low-density points as noise.

Pattern 3: Expectation–Maximization

Used in: GMM.

1. E-step: soft assignments.
2. M-step: update parameters.
3. Repeat until log-likelihood converges.

Common Pitfalls and How to Avoid Them

Pitfall 1: Shape Mismatches

```
X = np.array([[1,2],[3,4]])  # (2,2)
centroid = np.array([1.5,2.5])  # (2,)
diff = X - centroid  # works via broadcasting
# If centroids is (2,2) then X - centroids is element-wise and likely
    wrong.
diff = X[:, np.newaxis] - centroids  # (2,1,2) - (2,2) -> (2,2,2)
```

Pitfall 2: Empty Clusters

Check for empty clusters before computing means:

```
labels = np.array([0,0,1,1])
k = 3
for i in range(k):
    cluster_points = X[labels == i]
    if len(cluster_points) == 0:
        continue
    new_centroid = cluster_points.mean(axis=0)
```

Pitfall 3: Singular Covariance Matrices

Always regularize before inverting:

```
cov = np.cov(X.T)
cov_reg = cov + np.eye(d) * 1e-6
inv_cov = np.linalg.inv(cov_reg)
```

Pitfall 4: Forgetting to Return Values

```
def kmeans(X, k):
    # ... implementation ...
    labels = np.argmin(distances, axis=1)
    return labels, centroids
```

Pitfall 5: Modifying Outer Variables in Nested Functions

Use `nonlocal` when necessary:

```
def dbscan(X, eps, min_samples):
    cluster_id = 0
    def expand_cluster(point_idx, neighbors):
        nonlocal cluster_id
        cluster_id += 1
```

Quick Reference: Which Functions to Use Where

K-means++ Init `np.random.randint()`, `np.linalg.norm()`, `np.minimum()`, `np.random.choice(p=...)`

K-means `X[:,np.newaxis]`, `np.argmin(axis=1)`, `X[labels==i]`, `.mean(axis=0)`, `np.allclose()`

K-medoids `np.where(labels==i)[0]`, `points[:,np.newaxis]-points`, `np.sum(axis=1)`, `set()==set()`

WCSS `np.unique()`, `np.sum(np.square())`

DBSCAN `np.full(n,-1)`, `np.where(dist<=eps)[0]`, `np.concatenate()`, `nonlocal`

Gaussian PDF `np.linalg.inv()`, `np.linalg.det()`, `diff@inv`$_c ov * diff, np.eye() * 1e-6$

GMM EM `np.cov(X.T)`, `lik*weights`, `.sum(axis=1)[:,np.newaxis]`, `resp[:,k]@X`, `np.log()`, `np.sum()`

Good luck with your implementation! Remember: the goal is to understand how these algorithms work, not just to get code that runs.